Technical Architecture Specification: YouTube Media Converter
Document Version 1.0 | Apr 19, 2026 | Status: Draft

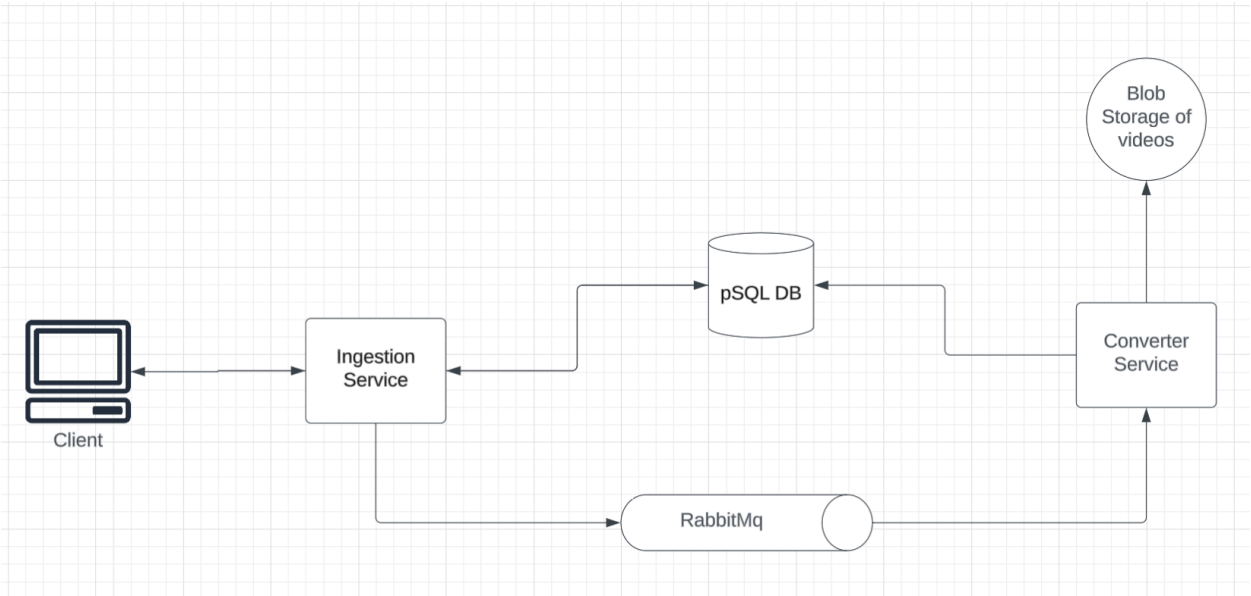
1. EXECUTIVE SUMMARY

This technical specification details the full-stack architectural design for an asynchronous YouTube-to-MP4/MP3 conversion system. The platform is engineered to manage high-concurrency processing through a decoupled message queue architecture, utilising PostgreSQL for persistent job state management and Railway Blob Storage for secure media hosting. All system components are containerised via Docker to ensure environment parity and are deployed on the Railway cloud platform.

Legal Compliance: The unauthorised retrieval of YouTube content may constitute a violation of the platform's Terms of Service. This architecture is strictly intended for personal use or for the processing of media where the user maintains explicit intellectual property rights.

2. ARCHITECTURAL DESIGN OVERVIEW

The system architecture comprises four primary service modules integrated with three distinct infrastructure components.



Service Components

- **React Frontend:** Serves as the primary user interface, facilitating URL submission, real-time conversion monitoring, and secure file retrieval.
- **Job Service (Java / Spring Boot):** Functions as the centralised API gateway. This service manages incoming requests, persists job records within PostgreSQL, dispatches messages to RabbitMQ, and provides comprehensive status and download endpoints.
- **Converter Service (Python / FastAPI):** Operates as an asynchronous worker node. It ingests jobs from RabbitMQ, utilises the yt-dlp library for media acquisition and transcoding, transfers final assets to blob storage, and synchronises job states in the PostgreSQL database.
- **RabbitMQ:** Acts as the message brokerage layer, effectively decoupling the Job and Converter services to ensure system resilience and support asynchronous processing workflows.

Infrastructure Components

- **PostgreSQL (Railway):** A relational database dedicated to the persistence of job-related metadata and operational statuses.
- **Blob Storage (Railway Buckets):** An S3-compatible object storage solution utilised for the secure hosting of processed MP4 and MP3 assets.
- **RabbitMQ Infrastructure:** Orchestrated as a containerised Docker service within the Railway cloud environment.

3. OPERATIONAL REQUEST WORKFLOW

1. The end-user provides a YouTube localised resource locator (URL) and designates the target output format (MP4 or MP3) via the React-based client interface.
2. The frontend application transmits a POST request to the /api/jobs endpoint of the Job Service, encapsulating the source URL and specified format metadata.
3. The Job Service initialises a persistent job record within the PostgreSQL database with a status of "PENDING," dispatches a task execution message to the RabbitMQ broker, and returns a unique job identifier to the client.
4. The Converter Service ingests the queued message, transitions the job state to "PROCESSING," and executes the media acquisition and transcoding process using the yt-dlp library.
5. Upon successful completion, the Converter Service transfers the generated asset to Railway Blob Storage and updates the PostgreSQL record to a "DONE" status, inclusive of the relevant file reference.

6. The client frontend performs periodic polling of the GET /api/jobs/{id}/status endpoint at four-second intervals; once the terminal "DONE" state is detected, the interface presents a download option powered by a secure, presigned blob URL.

4. FRONTEND ARCHITECTURE

Technical Stack

- React + Vite: Utilised as the primary user interface framework and build orchestration tooling.
- TanStack Router: Implements type-safe, file-based client-side navigation.
- TanStack Query: Manages asynchronous state and HTTP synchronisation. This replaces standalone libraries such as Axios, with all network requests handled via useQuery and useMutation hooks.
- ShadCN + Tailwind CSS: Provides a standardised component library and utility-first styling methodology.
- Vitest: Executes unit and component-level testing within a Vite-native environment.
- Cypress: Facilitates comprehensive end-to-end (E2E) testing suites.

User Interface States

- Idle: Default state featuring the URL input field and conversion initiation control.
- Submitting: Visual feedback provided via a loading indicator during the initial POST request.
- Processing: Displayed during the polling phase; includes the job identifier for user reference.
- Done: Transition to completion, revealing a download interface powered by a secure presigned URL.
- Failed: Error state offering diagnostic information and a retry mechanism.

Polling Methodology

State synchronisation is achieved through TanStack Query's refetchInterval, configured at 4000ms. To optimise network resource consumption, polling is programmatically terminated upon reaching a terminal status (DONE or FAILED).

Future Enhancements

The system may be upgraded to utilise Server-Sent Events (SSE) via Spring Boot's SseEmitter. This transition would enable real-time server push capabilities, providing a more efficient alternative to polling without the architectural overhead of WebSockets.

5. JOB SERVICE (JAVA)

Technology Stack

- Spring Boot 3: Utilised as the primary application framework.
- Spring AMQP: Facilitates RabbitMQ integration for dispatching jobs to the message broker.
- Spring Data JPA: Employs PostgreSQL ORM for the persistence of job-related data.
- AWS SDK v2 (S3 Client): Used to generate secure, presigned URLs for Railway Buckets (S3-compatible).
- JUnit 5: Provides the framework for comprehensive unit testing.
- Testcontainers: Facilitates integration testing via the orchestration of real PostgreSQL and RabbitMQ Docker containers.

Primary API Endpoints

- POST /api/jobs: Receives YouTube URLs and format specifications, initialises job records, publishes tasks to the queue, and returns unique job identifiers.
- GET /api/jobs/{id}/status: Provides real-time job status updates and supplies download URLs upon task completion.

Job Lifecycle States: PENDING → PROCESSING → DONE / FAILED

6. CONVERTER SERVICE (PYTHON)

Technology Stack

- Python: Selected as the primary language to leverage programmatic integration with the yt-dlp library.
- FastAPI: A high-performance web framework used for health monitoring endpoints.
- yt-dlp: Serves as the core engine for media retrieval and transcoding.
- Boto3: An S3-compatible client for transmitting processed assets to Railway Buckets.
- Pika: The RabbitMQ client responsible for consuming task messages from the brokerage layer.

Media Conversion Specifications

- MP3: Performs audio extraction and transcoding to MP3 format with configurable bitrates.
- MP4: Facilitates the acquisition of high-definition video and audio streams, followed by an automated merging process into MP4 format.

Upon task fulfillment, the service transmits the output file to Railway Bucket storage and synchronises the PostgreSQL job record with the corresponding file key and a terminal status of DONE.

7. INFRASTRUCTURE SPECIFICATIONS

RabbitMQ Broker

The messaging middleware is orchestrated as a containerised Docker service within the Railway environment. The Job Service acts as a producer, publishing tasks to a dedicated conversion queue, while the Converter Service operates as a consumer. System resilience is maintained through RabbitMQ's acknowledgement model, which facilitates the automated requeueing of failed job instances.

Relational Database (PostgreSQL)

A managed PostgreSQL instance on Railway serves as the primary data store. This component is responsible for the persistent storage of operational metadata, including unique job identifiers, source YouTube URLs, specified output formats, real-time lifecycle statuses, storage object keys, and associated temporal markers.

Object Storage (Railway Buckets)

Media assets are hosted via Railway's native S3-compatible object storage. Following successful transcoding, the Converter Service executes a secure upload to the bucket. The Job Service facilitates client-side retrieval by generating short-lived, presigned URLs, ensuring that all stored files remain private by default. The associated cost structure is equivalent to Cloudflare R2, providing unlimited API operations without additional egress fees.

8. DEPLOYMENT AND ORCHESTRATION

Local development environments are standardised through a comprehensive docker-compose.yml configuration, which orchestrates the frontend, job-service, converter-service, and rabbitmq components, alongside a local PostgreSQL instance. Production deployment involves individual Railway services within a unified project scope. Configuration management is achieved through Railway's variable reference system, which securely injects essential environment variables, such as database connection strings, RabbitMQ endpoints, and object storage credentials, across the service mesh.

9. COMPREHENSIVE TESTING STRATEGY

Frontend Framework

- Vitest: Employed for the execution of unit tests targeting custom hooks, utility functions, and UI components.
- Cypress: Utilised for end-to-end (E2E) testing to validate the integrity of the full conversion lifecycle, from initial URL submission to final asset retrieval.

Job Service (Java)

- JUnit 5: Facilitates isolated unit testing of core service logic and REST controllers.
- Testcontainers: Provides a robust integration testing environment by orchestrating ephemeral PostgreSQL and RabbitMQ instances within Docker.

Converter Service (Python)

- pytest: Used to perform unit and integration testing of the media conversion logic and queue consumption mechanisms.

10. ARCHITECTURAL SERVICE SUMMARY

Service	Technological Stack	Primary Responsibilities
React Frontend	React, Vite, TanStack	Interface management for request submission, status polling, and asset retrieval.
Job Service	Java, Spring Boot	API orchestration, job lifecycle management, and secure URL generation.
Converter Service	Python, FastAPI, yt-dlp	Asynchronous processing: media acquisition, transcoding, and storage synchronisation.
RabbitMQ	RabbitMQ (Railway)	Decoupled, asynchronous message brokerage.
PostgreSQL	Railway Postgres	Persistent storage for job state and system metadata.

Service	Technological Stack	Primary Responsibilities
Blob Storage	Railway Buckets (S3)	Secure hosting of processed media assets.